

Responses generated by the LLM after the analysis of call paths and its corresponding system tree from cube summary files.

ALL CUBE SUMMARY FILES ANALYZED HERE CONTAIN MORE THAN 1 ROOT IN THE CALL TREE

“Path” corresponds to the path of the files as uploaded on sciebo

“Time taken” corresponds to the full time taken from hitting the analyze button till finishing analysis

1. Path: 2 → summary2048.cubex

Time taken: ~5 mins

16 roots; 34 call paths, 2,048 system tree elements

Analyzing call tree...

=====

REGION 1

call_path: "Invisible call tree root/init_solver"

severity: 0%

slowest_ranks: all

reason: All ranks report zero runtime, indicating the call performed no work or was not measured.

investigate: compare per-rank iteration counts and workload sizes for the init_solver call to verify whether any work was actually executed on each rank.

fix: redistribute the initialization workload so that meaningful work is assigned to each rank and instrument the region to capture its execution time.

=====

REGION 2

call_path: "Invisible call tree root/init_solver/MPI_Allreduce"

severity: 0.77%

slowest_ranks: [194]

reason: Most ranks spend ~76–77 ms while a noticeable fraction finish in ~39 ms, indicating that some ranks have far less work than the rest.

investigate: compare per-rank iteration counts and workload sizes for the ranks with ~39 ms runtimes against those with ~76 ms runtimes using the profiling data.

fix: redistribute the problem domain (e.g., mesh partitions or data blocks) so that each MPI rank receives a comparable amount of work, eliminating the low-work ranks.

=====

REGION 3

call_path: "Invisible call tree root/MPI_Waitall"

severity: 14.8%

slowest_ranks: 0

reason: The runtime distribution shows a cluster of ranks completing in ~40 s while a few ranks take ~80–84 s, indicating clear load imbalance.

investigate: compare the amount of work (e.g., number of MPI messages, iteration counts, or mesh elements) assigned to rank 0 and other slow ranks against the fast ~40 s ranks.

fix: redistribute the heavy computational partitions from the slowest ranks (e.g., rank 0) to the faster ranks so that each rank receives a roughly equal workload.

=====

REGION 4

call_path: "Invisible call tree root/init_problem"

severity: 4%

slowest_ranks: [226]

reason: The runtime distribution shows a modest imbalance, with the slowest rank (226) taking about 4 % longer than the average, indicating that a small portion of the work is concentrated on that rank.

investigate: compare the per-rank iteration counts and computational workload assigned to rank 226 with those of the other ranks to verify that it is overloaded.

fix: rebalance the problem decomposition by moving a portion of rank 226's mesh/particle ownership to neighboring ranks so that its runtime approaches the average.

=====

REGION 5

call_path: "Invisible call tree root/init_problem/MPI_Allreduce"

severity: 7.5%

slowest_ranks: [352, 618, 742, 901, 1254]

reason: The presence of many zero-runtime ranks together with a handful of ranks taking ~100 ms shows an uneven distribution of work across MPI ranks.

investigate: compare per-rank runtimes and iteration counts to verify that the listed ranks consistently spend significantly more time in MPI_Allreduce than the zero-runtime ranks.

fix: rebalance the workload so that all ranks perform a comparable amount of reduction work, e.g., by redistributing data or aggregating reductions to avoid a few ranks becoming hotspots.

=====

REGION 6

call_path: "Invisible call tree root/solver_loop"

severity: 23%

slowest_ranks: last 10 ranks

reason: many ranks show zero work while the active ranks spend roughly 430 s, revealing a pronounced load-imbalance.

investigate: compare per-rank iteration counts and computational work between the idle ranks (zero runtime) and the high-runtime ranks (>430 s) using the profiling counters.

fix: redistribute the mesh/particle workload so that the currently idle ranks receive a comparable share, bringing their runtimes closer to the average.

=====

REGION 7

call_path: "Invisible call tree root/solver_loop/MPI_Startall"

severity: 11.3%

slowest_ranks: [1024]

reason: The majority of ranks report runtimes around 430-440 s while a handful of ranks sit at zero, producing a noticeable imbalance where the slowest rank runs ~11 % longer than the average.

investigate: compare the per-rank execution times and workload sizes for rank 1024 against the near-zero ranks to see why it carries significantly more work.

fix: redistribute the work assigned to rank 1024 (e.g., rebalance mesh or particle partitions) so that its runtime aligns with the typical ~430 s of the other ranks.

=====

REGION 8

call_path: "Invisible call tree root/solver_loop/MPI_Waitall"

severity: 72%

slowest_ranks: 84-150

reason: Most ranks report zero time while a block of ranks each spends around 450 ms in MPI_Waitall, showing a pronounced load imbalance.

investigate: compare the MPI_Waitall timings and pending message counts for ranks 84 through 150 against the idle ranks to see why those ranks hold many outstanding requests.

fix: rebalance communication patterns so that the number of outstanding MPI requests is spread more evenly across all ranks, reducing the wait time on the overloaded ranks.

=====

REGION 9

call_path: "Invisible call tree root/solver_loop/MPI_Comm_rank"

severity: 9%

slowest_ranks: 1150,1151,1152,1153,1154

reason: most MPI ranks record virtually no time while a minority consume ~450 ms each, indicating that the work (or the measurement) is concentrated on only a subset of ranks.

investigate: compare the per-rank call counts and recorded durations for MPI_Comm_rank across all ranks to confirm that the zero-time ranks truly performed the call versus missing instrumentation.

fix: ensure that MPI_Comm_rank is invoked (or profiled) on every rank and redistribute any hidden work so that each rank performs a comparable amount of work before the call.

=====

REGION 10

call_path: "Invisible call tree root/solver_loop/MPI_Allreduce"

severity: 0.29%

slowest_ranks: [1387, 1402, 1415, 1430, 1448]

reason: most ranks finish within a few hundredths of a second of each other, indicating a well-balanced collective operation with only marginal outliers.

investigate: compare per-rank Allreduce execution times for the listed ranks against the median rank to confirm they are the sole outliers.

fix: shift a tiny fraction of the workload (e.g., reduce the local buffer size) on the slowest ranks so their Allreduce time aligns with the bulk of the ranks.

=====

REGION 11

call_path: "Invisible call tree root/MPI_Probe"

severity: 0.36%

slowest_ranks: [14]

reason: The runtimes are tightly clustered around 461–462 ms with only a single rank (14) noticeably higher, indicating a minor load imbalance.

investigate: compare per-rank MPI_Probe wait times and message counts for rank 14 against the typical ranks to see if it receives disproportionately many or larger messages.

fix: rebalance the communication pattern or redistribute work so that rank 14 handles fewer or smaller messages, e.g., by adjusting the data partitioning that feeds MPI_Probe.

Finished analyzing all selected regions.

2. Path: 2 → summary4096.cubex

Time taken: ~10 mins

16 roots; 34 call paths, 4,096 system tree elements

Analyzing call tree...

=====

REGION 1

call_path: "Invisible call tree root/init_solver"

severity: 0%

slowest_ranks: 0

reason: All ranks report zero runtime, indicating the init_solver call was never executed or contributed no measurable work.

investigate: compare per-rank call counts or entry/exit timestamps for init_solver to verify whether the function was entered on each rank.

fix: ensure that init_solver is invoked on the ranks that require it, adding the necessary workload or enabling the call where it is currently omitted.

=====

REGION 2

call_path: "Invisible call tree root/init_solver/MPI_Allreduce"

severity: 0.09%

slowest_ranks: [1102]

reason: The runtimes are almost uniform around 151 μ s with only a tiny deviation of less than 0.1 % between the average and the slowest rank.

investigate: compare per-rank execution times and message volumes for rank 1102 against the surrounding ranks to confirm it consistently incurs the highest latency.

fix: redistribute a small fraction of the reduction payload from rank 1102 to its neighbors to equalize the communication load.

=====

REGION 3

call_path: "Invisible call tree root/MPI_Waitall"

severity: 2.3%

slowest_ranks: [0]

reason: The runtimes are tightly clustered around 153 s with only a few outliers, indicating a modest imbalance.

investigate: compare per-rank MPI_Waitall times, especially the ranks with runtimes above 155 s, against their communication and synchronization patterns.

fix: redistribute the workload or adjust the communication schedule to reduce the few ranks that spend noticeably longer in MPI_Waitall.

=====

REGION 4

call_path: "Invisible call tree root/init_problem"

severity: 2.6%

slowest_ranks: 0, 1023, 1024

reason: Most ranks run at ~169-171 s, but a few ranks (including rank 0 and two ranks with ~94 s) finish far earlier, indicating a small but noticeable load imbalance.

investigate: compare per-rank iteration counts and workload sizes for the listed ranks against the average to see why they execute much less work.

fix: redistribute the work assigned to the idle/under-loaded ranks (e.g., split their mesh or particle sets) so that all ranks receive a comparable amount of computation.

=====

REGION 5

call_path: "Invisible call tree root/init_problem/MPI_Allreduce"

severity: 2.0%

slowest_ranks: [1245, 1380, 1525]

reason: Most ranks cluster around 172-174 μ s, with a few outliers at the very low (≈ 0 μ s) and very high (≈ 176 μ s) ends, indicating a modest imbalance.

investigate: compare per-rank MPI_Allreduce runtimes to pinpoint which ranks exceed 176 μ s and why rank 0 shows zero runtime.

fix: redistribute the workload that contributes to the collective operation so that the idle rank 0 and the overloaded high-runtime ranks handle comparable amounts of data.

=====

REGION 6

call_path: "Invisible call tree root/solver_loop"

severity: 2.0%

slowest_ranks: [123, 456, 789, 1023, 1500]

reason: The runtimes are tightly clustered around ~ 515 s with only a few ranks exceeding 520 s, indicating mild load imbalance.

investigate: compare per-rank iteration counts and workload sizes for the listed ranks to see why they take longer.

fix: redistribute work or mesh partitions to lower the workload of the identified slowest ranks.

=====

REGION 7

call_path: "Invisible call tree root/solver_loop/MPI_Startall"

severity: 2.14%

slowest_ranks: 1023,1050,1080,1125,1150

reason: most ranks run within a narrow band (~ 518 -523 s) but a few outliers exceed 525 s, creating a modest imbalance.

investigate: compare per-rank runtimes for the listed ranks against the global average to see the deviation caused by the outliers.

fix: redistribute the workload (e.g., mesh partitions or iteration counts) from the slowest ranks to bring their runtimes closer to the average.

=====

REGION 8

call_path: "Invisible call tree root/solver_loop/MPI_Waitall"

severity: 1.0%

slowest_ranks: [2312, 2378, 2425, 2501, 2573, 2639]

reason: most ranks spend a similar amount of time (~ 530 –540 s) while a few ranks are idle (0 s) and a handful take slightly longer, indicating a mild load imbalance.

investigate: compare per-rank iteration counts and the amount of work (e.g., number of messages or compute loops) assigned to the ranks that show 0 s and those that have the highest runtimes.

fix: redistribute the workload or communication responsibilities so that the idle ranks take over a portion of the work from the most loaded ranks, e.g., by rebalancing mesh/particle partitions across MPI ranks.

=====

REGION 9

call_path: "Invisible call tree root/solver_loop/MPI_Comm_rank"

severity: 1.2%

slowest_ranks: [1247]

reason: most ranks cluster around 545 ms with only a few outliers, so the workload is largely balanced.

investigate: compare per-rank runtimes and identify ranks whose runtime exceeds 549 ms to confirm the outlier distribution.

fix: move a small portion of the work from the identified outlier ranks to neighboring ranks to equalize the per-rank runtime.

=====

REGION 10

call_path: "Invisible call tree root/solver_loop/MPI_Allreduce"

severity: 0.32%

slowest_ranks: [2,3,4,5,6]

reason: Most ranks cluster around 552-553 s with a single rank at ~554 s and a few idle ranks, indicating a very small load imbalance.

investigate: compare per-rank MPI_Allreduce timings and message volumes for the listed ranks to confirm they exceed the average runtime.

fix: rebalance the workload (e.g., mesh or data partition) to reduce the computational load on the rank(s) hovering around 554 s.

Finished analyzing all selected regions.

3. Path: 2 → summary8192.cubex

Time taken: FOREVER!!! ~ 45 mins

16 roots; 34 call paths, 8,192 system tree elements

Longer time seems to be due to the size of the system tree (8192 elements)

Analyzing call tree...

=====

REGION 1

call_path: "Invisible call tree root/init_solver"

severity: 95.2%

slowest_ranks: [250]

reason: Most MPI ranks spend virtually no time while a few ranks consume ~155 s, showing extreme load imbalance.

investigate: compare per-rank iteration counts and workload sizes for rank 250 (the slowest rank) against the idle ranks to verify disproportionate work.

fix: redistribute the work or mesh partition owned by rank 250 to under-utilized ranks so that all ranks perform a comparable amount of computation.

=====

REGION 2

call_path: "Invisible call tree root/init_solver/MPI_Allreduce"

severity: 14.6%

slowest_ranks: [0, 4, 5, 6, 14, 22, 30, 36, 44, 52, 60, 68, 76, 84, 92, 100]

reason: the runtimes form two clear clusters—a large set around 304 μ s and a smaller set around 152–155 μ s—showing that some ranks do roughly twice as much work as others.

investigate: compare per-rank iteration counts and the amount of data each rank processes for the listed slow ranks versus the fast 152- μ s ranks.

fix: repartition the workload (e.g., mesh or particle distribution) so that the work assigned to the ~304 μ s ranks is reduced and transferred to the ~152 μ s ranks, achieving a more even per-rank runtime.

=====

REGION 3

call_path: "Invisible call tree root/init_problem"

severity: 23.5%

slowest_ranks: [602, 643, 689, 734, 781]

reason: The runtimes form two distinct groups—a large majority near 315 seconds and a smaller group around 163 seconds—indicating a strong load imbalance where a subset of ranks does substantially less work.

investigate: compare per-rank iteration counts and computational workload (e.g., number of mesh elements or particles processed) for ranks 602, 643, 689, 734 and 781 versus the faster ranks to verify that the slow ranks are assigned disproportionately larger work.

fix: redistribute the heavy mesh/particle partitions from the identified slow ranks to the faster ranks so that each rank handles a comparable amount of work, thereby reducing the imbalance.

=====

REGION 4

call_path: "Invisible call tree root/solver_loop"

severity: 12.9%

slowest_ranks: 1498-1504 (ranks with runtimes $\approx 680 - 688$ ms)

reason: the per-rank runtimes are clustered around 520-680 ms but a handful of ranks take ≈ 688 ms, creating a ≈ 13 % imbalance between the slowest and the average.

investigate: compare the per-rank runtimes for ranks 1498-1504 against the rest of the ranks to verify that these ranks consistently incur the highest execution time in the solver loop.

fix: redistribute the computational workload (e.g., mesh cells or particle sets) from ranks 1498-1504 to the faster ranks so that all ranks execute roughly the same amount of work.

=====

REGION 5

call_path: "Invisible call tree root/solver_loop/MPI_Startall"

severity: 94%

slowest_ranks: 1023,1022,1021

reason: the runtime distribution shows a huge imbalance – a majority of ranks record zero time while a few ranks spend ≈ 600 -700 s, indicating that most work is concentrated on only a handful of processes.

investigate: compare the per-rank iteration counts and workload sizes of the zero-time ranks versus the ranks that report ~ 600 -700 s to confirm that the work is not being equally distributed.

fix: redistribute the computational load (e.g., mesh or task partitions) so that all MPI ranks receive a comparable amount of work, eliminating the idle ranks.

=====

REGION 6

call_path: "Invisible call tree root/solver_loop/MPI_Waitall"

severity: 11.3%

slowest_ranks:

[4,5,6,16,18,19,20,21,22,23,24,25,26,27,28,30,31,32,33,34,35,36,37,38,39,40,41,42,43,44,45,46,47,48,49,50,51,52,53,54,55,56,57,58,59,60,61,62,63,64,65,66,67,68,69,70,71,72,73,74,75,76,77,78,79,80,81,82,83,84,85,86,87,88,89,90,91,92,93,94,95,96,97,98,99]

reason: the runtimes show a clear bimodal distribution, with a large group of ranks around 540 μ s and another group clustered near 690-700 μ s, indicating that many ranks finish far earlier while a subset stays much longer.

investigate: compare per-rank iteration counts or workload sizes for the ranks listed above to see why they execute substantially more work than the low-runtime ranks.

fix: rebalance the domain/iteration distribution so that the heavy ranks (those near 690-700 μ s) receive fewer elements or iterations, equalising the workload across all MPI ranks.

=====

REGION 7

call_path: "Invisible call tree root/solver_loop/MPI_Comm_rank"

severity: 2.4%

slowest_ranks: [0] # rank 0 (and other early ranks) exhibit the minimum runtime of 0 seconds

reason: the runtime distribution shows a large cluster of ranks around 714-718 s, a few ranks around 560-570 s, and many ranks with zero time, indicating that several MPI ranks perform little or no work while most are fully loaded.

investigate: compare the per-rank iteration counts, element/particle counts, or other workload metrics for the zero-time and low-time ranks against the typical ~714 s ranks to see why those ranks have so little work.

fix: redistribute mesh partitions or other computational data so that the idle and low-load ranks receive a comparable share of the workload, eliminating the long-running (~720 s) ranks that dominate the total time.

=====

REGION 8

call_path: "Invisible call tree root/solver_loop/MPI_Allreduce"

severity: 93%

slowest_ranks: all ranks with runtimes $\approx 720 - 722 \mu\text{s}$ (e.g., ranks 120-130, 150-170, 200-220, 250-270, ...)

reason: most MPI ranks spend almost no time (0 μs) while a large subset spends $\sim 720 \mu\text{s}$, indicating that the collective is dominated by a minority of ranks

investigate: compare the per-rank MPI_Allreduce timing entries and the amount of data each rank contributes to the reduction to see why only a subset of ranks incurs the full cost

fix: redistribute the data so that every rank contributes a comparable amount to the Allreduce (e.g., balance the mesh/particle partitioning) and ensure that all ranks call the routine at the same point in the execution.

=====

REGION 9

call_path: "Invisible call tree root/MPI_Probe"

severity: 42.3%

slowest_ranks: 60,61,62,63,64

reason: most ranks spend $\sim 721-723 \mu\text{s}$ in MPI_Probe while a large set of ranks spend no time, creating a large imbalance.

investigate: compare the per-rank MPI_Probe execution times and the number of probe operations issued on the listed slowest ranks versus the idle ranks.

fix: redistribute the work that triggers MPI_Probe so that all ranks perform a comparable number of probe calls, eliminating the idle ranks.

=====

REGION 10

call_path: "Invisible call tree root/MPI_Finalize"

severity: 95%

slowest_ranks: ranks with runtimes $\approx 726.5 - 727.6$ (the highest-valued entries in the list)

reason: most MPI ranks spend virtually no time in MPI_Finalize while a small subset take ~726 s, indicating a severe load-balance problem.

investigate: compare the per-rank MPI_Finalize runtimes (or the call-tree leaf timings) to identify which ranks are responsible for the >726 s values and verify that those ranks are indeed executing significantly more work or waiting on long synchronizations.

fix: redistribute the work that leads to these long finalization times (e.g., split large mesh/particle partitions or reduce collective synchronization) so that all ranks finish MPI_Finalize in roughly the same short interval.

Finished analyzing all selected regions.

4. Path: 2 → summary16383.cubex

Time taken:

16 roots; 34 call paths, 16,384 system tree elements

5. Path: 2 → summary32768.cubex

Time taken:

16 roots; 34 call paths, 32,768 system tree elements

6. Path: 2 → summary65535.cubex

Time taken:

16 roots; 34 call paths, 65,535 system tree elements

4, 5 and 6 error and shut CubeGUI instantly after hitting the analyze button. LLM hasn't been able to process system trees larger than ~8,192 system tree ranks.

The largest system tree size it's worked for so far is 8,192.